From Signals to Sockets: How Networks Work, Explained for C Programmers

Made by AI

***Abstract*—Computer networking is usually taught as a stack of categories to memorize: seven OSI layers, "Layer 2" and "Layer 3" devices, TCP versus UDP, a long list of protocols. This tutorial instead derives the structure of networks from a few physical and engineering facts, and then shows what those facts mean for a programmer writing network code in C. It begins with the channel, deriving rather than merely stating why a signal of limited bandwidth, disturbed by thermal noise and weakened by distance, can carry bits only at a bounded rate and never without error, and distinguishing the capacity of a channel from the entropy of a source. It explains why links are shared by switching packets rather than circuits, why protocols are organized in layers, and why the Internet Protocol forms a deliberately minimal "narrow waist" that every network can carry. It explains how addresses, routes, and names work, from prefixes and longest-match forwarding to BGP and the Domain Name System. It then presents the end-to-end argument and the transport protocols built on it, the Berkeley sockets interface through which C programs reach the network, the readiness and completion interfaces that let one server handle many connections, and what TLS does and does not guarantee. A section derives the meaning of commonly confused terms from the structure they describe. Finally, it gives canonical forms for network code, each illustrated by a small C program run over the loopback interface: byte order, wire formats, the absence of message boundaries in TCP, partial transfers, the latency cost of small writes, address families, server restarts, datagram limits, round trips, error handling, untrusted input, and verifying a TLS peer's identity. A complete client and server written in these forms, and tested against a hostile client, closes the practical part.**

***Index Terms*—Berkeley sockets, DNS, end-to-end argument, event-driven I/O, Internet Protocol, layering, network programming, routing, TCP, UDP.**

I. Introduction

A programmer who opens a socket uses, without seeing it, a chain of mechanisms that reaches from a voltage on a wire or a radio wave in the air to a call such as `recv()`. Textbooks describe that chain as a set of layers and protocols to be learned by name, and it is easy to come away knowing the names without knowing why the chain has the shape it has, or what it promises to the program at its top.

Manuscript prepared November 2026.

This paper is the third in a series that tries to replace memorization with derivation. The first explained the Rainbow Series of trusted-systems guidance, and the second explained C as a portable assembly language over a minimal set of machine operations. Here the subject is the network. The structure of the argument is the same. Physical media diverge widely; engineers introduced categories to describe the differences; a small common core makes it possible to connect them all; a portable programming interface sits on that core; and programs are correct only when written in canonical forms that rely on nothing the core does not promise.

Section II begins with the channel and its limits. Section III explains why links are shared by packets. Section IV derives layering, and Section V the role of the Internet Protocol as the network's minimal common core. Section VI explains addresses and routing, and Section VII names. Section VIII presents the end-to-end argument and the transport protocols. Section IX introduces the sockets interface, Section X shows how one server handles many connections, Section XI explains TLS, Section XII derives commonly confused terms, Section XIII the canonical forms of network code, each demonstrated by a short C program run on Linux over the loopback interface, and Section XIV combines them in a complete client and server. Section XV discusses what the paper leaves out, Section XVI concludes, and the Appendix lists tools for observing the network directly.

II. The Channel: Signals, Capacity, and Errors

Every network rests on a physical channel: a pair of copper wires, an optical fibre, or a band of radio spectrum. The channel does not carry bits. It carries a signal, a voltage, a light intensity, or an electromagnetic wave that varies in time, and bits must be encoded into the signal at one end and recovered from it at the other.

It is natural to ask why that should impose any limit at all. If communication is simply an emitter changing a signal and a receiver observing it, why can the emitter not change the signal faster, or encode more bits in each change, without bound? Two physical facts answer the question, and a third explains why distance matters.

A. The Channel Cannot Change Arbitrarily Fast

No physical medium follows its input instantly. A copper pair has capacitance and inductance that smooth out rapid

changes; an optical fibre spreads a short pulse as it travels; a radio transmitter is permitted to occupy only a limited band of frequencies [1]. In each case the channel passes only a range of frequencies, and the width of that range is its *bandwidth* B, measured in hertz.

A band-limited signal has a remarkable property. Shannon stated it in 1949: "if a function $f(t)$ contains no frequencies higher than W cps, it is completely determined by giving its ordinates at a series of points spaced $1/2W$ seconds apart" [2]. The result, now called the sampling theorem, had been found earlier by Whittaker and by Kotel'nikov [3]. Its consequence for communication is direct. A signal confined to bandwidth B is fully described by $2B$ numbers per second, so the channel can carry at most $2B$ *independent* values per second; any faster variation would contain frequencies the channel does not pass. This is the limit implied by Nyquist's 1928 analysis of telegraph transmission [4].

B. Noise Limits What Each Value Can Say

If the receiver could measure each value exactly, a single value could carry unlimited information: the emitter could send 0.1, 0.11, or 0.111 volts and the receiver could tell them apart. What prevents it is *noise*. Every conductor above absolute zero produces a random voltage from the thermal agitation of its electrons, as Johnson measured and Nyquist explained in 1928 [5], [6]. Its power is proportional to temperature and bandwidth, kTB, which at room temperature is about -174 dBm in each hertz of bandwidth [7]; a 20 MHz channel therefore carries about -101 dBm of thermal noise before any other disturbance is counted.

Noise blurs every value the receiver measures, so the emitter can use only levels spaced far enough apart that the blur around one does not reach the next, as Fig. 1 illustrates. Hartley had proposed in 1928 to measure information by "the logarithm of the number of possible symbol sequences" [8]. Shannon estimated how many levels survive the noise: "the number of amplitudes that can be reasonably well distinguished is $K\sqrt{(P+N)/N}$ where K is a small constant in the neighborhood of unity", with P the signal power and N the noise power [2]. Each value therefore carries about $\log_2 \sqrt{1+S/N} = \frac{1}{2}\log_2(1+S/N)$ bits, and $2B$ values per second give

$$C = B \log_2\left(1 + \frac{S}{N}\right) \text{ bits per second.} \tag{1}$$

The heuristic gives the right formula, but Shannon himself called it a "crude estimate" [2]. His real achievement, in 1948, was to prove that the formula is exact in a strong sense: below the capacity C, coding exists that makes the error rate as small as desired, while above it "it is not possible to transmit at a higher rate by any encoding system without a definite positive frequency of errors" [9], [10]. In his 1949 paper he gave a geometric picture of why: each possible message, disturbed by noise, occupies a small sphere in a space of very many dimensions; the number of such noise spheres that fit, without overlapping, inside the sphere of all possible received signals bounds the number of messages that can be told apart, and C is the logarithm of that number per second [2]. Modern textbooks develop the theory in full [11], [12].

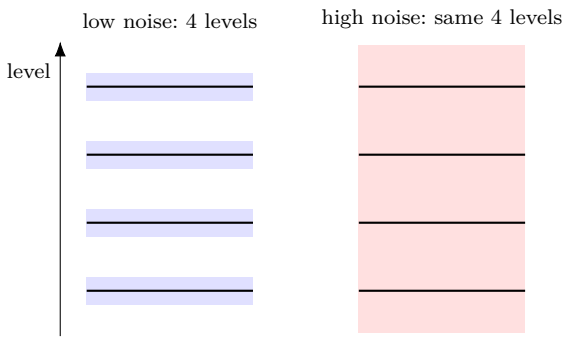


Fig. 1. Why noise limits the bits per symbol. Each line is a signal level and each band the spread that noise adds at the receiver. With low noise (left) the bands are separate and every level can be recognized; with high noise (right) they overlap, and fewer, more widely spaced levels must be used.

A worked example fixes the scale. A telephone-grade channel of about 3 kHz with a signal-to-noise ratio of 30 dB ($S/N = 1000$) can carry at most about 30 kbit/s, whatever the modem. Faster links are faster because they have more bandwidth, a better signal-to-noise ratio, or both.

C. Why Distance Matters

Distance enters through the signal-to-noise ratio. The signal weakens as it travels, while the noise at the receiver does not. In a cable or fibre the power falls exponentially with length, by a fixed number of decibels per kilometre: silica fibre loses about 0.2 dB/km near a wavelength of 1550 nm, as first demonstrated in 1979 [13], and the standard single-mode fibre specification allows up to 0.3 to 0.35 dB/km [14]; at 0.2 dB/km, only about one percent of the light remains after 100 km. In free space, the received power falls with the square of the distance [15]. Either way, S/N falls with distance, and capacity falls with it. That is why cable standards specify maximum lengths, why twisted-pair Ethernet is limited to about 100 m [16], and why long links need amplifiers or regenerators.

Distance also costs time, independently of rate. A signal in optical fibre is delayed by about 5 microseconds per kilometre [17], roughly one and a half times the 3.3 microseconds per kilometre of light in vacuum, so a message crossing 4000 km of fibre takes at least 20 ms however fast the link. Distance therefore costs twice: it lowers the achievable rate, and it adds delay that no increase in rate can remove.

D. Entropy Is Not Capacity

Shannon's name is attached to two different limits that are easy to confuse. *Capacity* belongs to the channel: how many bits per second it can carry. *Entropy* belongs to the source: how much information its messages actually contain. For a source that emits symbols with probabilities

p_i, the entropy is $H = -\sum_i p_i \log_2 p_i$ bits per symbol [9]. A fair coin has $H = 1$ bit per toss, but a coin that lands heads 90 percent of the time has $H \approx 0.47$ bits, because its outcomes are predictable. Entropy bounds how far a message can be compressed, and capacity bounds how fast it can be sent; Shannon showed that one cannot "transmit at an average rate greater than C/H" symbols per second [9]. Compression and transmission are therefore separate problems, which is why networks can treat every message simply as bits.

E. What the Physics Means for Software

Three consequences reach all the way up to the program. First, *rate is finite*: a link has a capacity, and sending faster than it only fills queues. Second, *delay is unavoidable*, and grows with distance. Third, *errors are inevitable*: Shannon's theorem promises arbitrarily small error, not zero error, and real links occasionally deliver corrupted bits. Every frame therefore carries a check value computed from its contents, on Ethernet a 32-bit cyclic redundancy check [18], and a frame that fails the check is discarded.

F. The Media Diverge

Physical media make very different trade-offs, and Table I shows how far apart they are, using common Ethernet and Wi-Fi variants. Copper is cheap and easy to install but limited to about 100 m; fibre carries far higher rates over kilometres because its attenuation is so low; radio needs no cable at all but is shared by everyone in range and suffers interference and fading, so its peak rate is shared and rarely reached.

The table holds the same lesson as the cross-architecture tables of the C paper. Below the link layer everything differs: the medium, the signalling, the rate, and the reach. Above it, the same Ethernet frame format, with its addresses and check value, is carried over every wired variant [18]. A network that spans all of these must work with the weakest promises any of them can make.

Key point. A channel cannot change faster than its bandwidth allows, and noise limits how many levels each change can distinguish; together they fix the capacity $B \log_2(1 + S/N)$. Distance lowers the signal-to-noise ratio and adds delay. Everything above the channel must live with a bounded rate, a delay, and occasional errors.

III. Sharing the Channel: Circuits, Packets, and Frames

A dedicated wire between every pair of computers is impossible, so channels must be shared. The telephone network shared them by *circuit switching*: a path is reserved end to end for the duration of a call, whether or not anyone is speaking. Computer traffic is bursty, so a reserved circuit would sit idle most of the time.

A. Packet Switching

In the early 1960s Baran proposed a different design for a communications network that could survive damage: break messages into small blocks, and route each block independently through a mesh of nodes [24]. Independently, Davies at the National Physical Laboratory in Britain conceived the same idea in 1965, and in a 1966 proposal gave the blocks the name *packets* [25], [26]. Packet switching shares a link statistically: packets from many conversations take turns, so a link is busy whenever anyone has something to send. The price is that packets may wait in queues, may arrive out of order, and may be dropped when a queue overflows.

B. Frames on a Shared Medium

Within a single local network the same idea appears as the *frame*. Ethernet, as described by Metcalfe and Boggs in 1976, connected many computers to one shared cable [27]. Each frame carried the address of its destination, every station listened, and a station that detected another transmission waited and tried again. Modern Ethernet replaces the shared cable with switches that forward each frame only toward its destination, but the frame, with its addresses and its check value, remains.

C. Sharing the Air

Radio changes the sharing problem. A wired Ethernet station can detect a collision while it is transmitting, but a radio cannot reliably hear another transmission over its own [1], and two stations may each be in range of an access point without being in range of each other, the *hidden terminal* problem [28]. Wi-Fi therefore uses collision *avoidance* rather than detection: a station waits for the medium to be idle, then for a random additional interval, before transmitting, and relies on acknowledgments to learn whether a frame arrived [29], [1]. The same layer that offered a nearly error-free service on fibre must, on radio, retransmit routinely, which is one reason wireless links show more variable delay.

Key point. Bursty traffic is best served by sharing links packet by packet. The cost is queueing delay, reordering, and loss, which higher layers must handle.

IV. Why the Layers

The jobs described so far are different in kind. Turning bits into signals depends on the medium. Delivering a frame within one network depends on the network's own addressing. Delivering a packet across many networks needs addresses that span them all. Delivering a reliable stream to an application needs sequence numbers and retransmission. Engineers organized these jobs into *layers*, each using the service of the one below and offering a service to the one above.

TABLE I
REPRESENTATIVE LINK TECHNOLOGIES

Medium	Standard	Rate	Reach
Twisted pair, Category 5	1000BASE-T [19]	1 Gb/s	100 m
Twisted pair, Category 6A	10GBASE-T [20]	10 Gb/s	100 m
Multimode fibre (OM3)	10GBASE-SR [21]	10 Gb/s	300 m
Single-mode fibre	10GBASE-LR [21]	10 Gb/s	10 km
Single-mode fibre, four wavelengths	100GBASE-LR4 [22]	100 Gb/s	10 km
Radio, 2.4, 5, or 6 GHz bands	IEEE 802.11ax (Wi-Fi 6) [23]	about 9.6 Gb/s peak*, shared	varies with environment

*The figure commonly quoted by vendors: eight spatial streams at about 1.2 Gb/s each in a 160 MHz channel with the densest modulation. Real throughput is far lower and is shared by all stations.

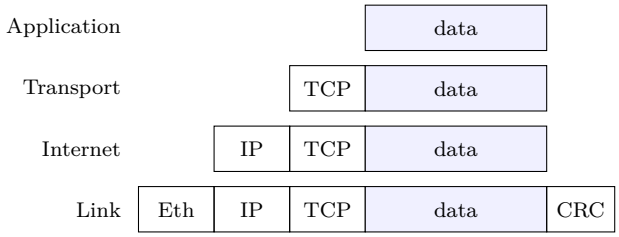


Fig. 2. Encapsulation. Each layer prepends its own header; the link layer also appends a check value. Only the matching layer at the receiver, or a device operating at that layer, reads each header.

A. Encapsulation

Layering is visible in every packet. Each layer adds its own header in front of the data handed down from above, a process called *encapsulation*, as shown in Fig. 2. A web page travels inside a TCP segment, inside an IP packet, inside an Ethernet frame. Each header is read only by the matching layer at the other end, or by intermediate devices that operate at that layer.

B. Two Models, One Idea

Two layer models are commonly taught. The OSI reference model, standardized by ISO, defines seven layers: physical, data link, network, transport, session, presentation, and application [30], [31]. The Internet's own host requirements describe a simpler stack of link, internet, transport, and application layers [32]. The difference is less important than it seems. The four Internet layers correspond to the OSI data link, network, transport, and application layers, with the physical medium below and OSI's session and presentation functions folded into applications.

The familiar device categories follow directly. A device that forwards frames within one network by their link-layer addresses is a "Layer 2" device, a *switch*. A device that forwards packets between networks by their IP addresses is a "Layer 3" device, a *router*. The numbers are OSI layer numbers, and nothing more: they say which header the device reads.

Key point. Layers separate jobs that differ in kind. "Layer 2" and "Layer 3" simply name which header a device reads to make its decision.

V. The Narrow Waist: IP as the Common Core

By the early 1970s there were already several incompatible packet networks, built on different media with different frame formats. Cerf and Kahn proposed a way to connect them without changing any of them: a common packet format and addressing scheme, carried over each network as ordinary data, with gateways between networks [33]. That design became the Internet Protocol (IP) and TCP.

A. What IP Promises

IP's promise is deliberately weak. It delivers individual packets, called datagrams, toward an address, and in the words of its specification "does not provide a reliable communication facility": there are no acknowledgments, no retransmissions, and no flow control [34]. The service is today called *best effort*: a datagram may be lost, duplicated, delayed, or delivered out of order, and IP makes no attempt to prevent it. Each datagram carries a source and destination address, and routers forward it hop by hop toward its destination. Because links have different maximum frame sizes, IP also defines a minimum that every host must handle: 576 bytes for IPv4 [34] and a minimum link size of 1280 bytes for IPv6 [35].

Clark later explained why the promise is so weak. The top goal of the Internet architecture was to interconnect existing networks, and the most important secondary goal was that communication should continue despite the loss of networks or gateways [36]. A minimal service could be implemented over any network at all, and keeping no per-conversation state in the routers meant that a failed router lost nothing that the endpoints could not recover.

B. The Hourglass

The result is often drawn as an hourglass, Fig. 3: many applications and transport protocols above, many link technologies below, and a single protocol at the narrow waist [37]. The waist is the network's equivalent of the minimal machine of the C paper in this series: the one thing every network agrees to carry, kept small precisely so that everything above and below can vary.

The waist has been widened once. IPv4's 32-bit addresses proved too few, and IPv6 replaced them with 128-bit addresses and a simplified header [35]. The two coexist today, which, as Section XIII shows, affects how portable programs must be written.

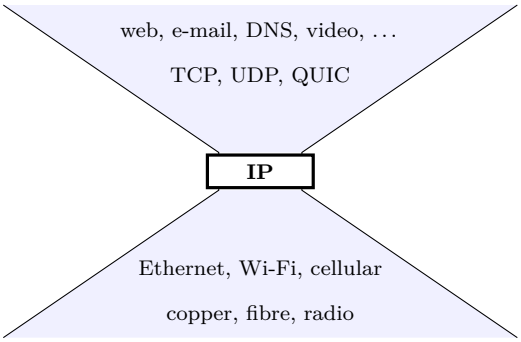


Fig. 3. The protocol hourglass. Diversity above and below a single, deliberately minimal protocol at the waist.

Key point. IP promises little, so that it can run over anything. Like the minimal machine beneath C, its smallness is the source of its universality.

VI. Addresses, Routes, and Paths

IP promises to carry a datagram toward an address. That promise raises three practical questions: how addresses are organized, how a router decides where to send a packet next, and how the routers learn enough to decide.

A. Addresses and Prefixes

An IPv4 address is 32 bits; an IPv6 address is 128. Addresses are not assigned one by one but in blocks that share a common leading part, the *prefix*, written with its length: `192.168.1.0/24` is the block of addresses whose first 24 bits match. This scheme, Classless Inter-Domain Routing (CIDR), lets a whole block be described, and routed, as a single entry [38]. When several entries match a destination, a router uses the most specific one, the *longest matching prefix* [38]. A routing table can then hold a few broad routes and a few narrow exceptions instead of one entry per host.

B. Private Addresses and NAT

Some blocks are reserved for private use and are not to be routed on the public Internet: `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16` [39]. Many home and office networks use them, and connect to the Internet through *network address translation* (NAT): a gateway rewrites the private source address and port of each outgoing packet to its own public address and a port it chooses, and reverses the rewriting for replies [40]. NAT stretched IPv4's limited address space a long way, but at a cost: hosts behind it cannot easily be reached from outside, because the gateway has no entry for a connection that did not start inside [41]. This is why the end-to-end model of Section VIII is weaker in practice than in principle, and one of the motivations for IPv6's vast address space.

C. From IP Address to Link Address

To deliver a packet on a local network, a host needs the link-layer address of the next machine, not just its IP address. On IPv4 networks the Address Resolution Protocol (ARP) finds it by broadcasting a question ("who has this IP address?") and caching the answer [42]; IPv6 uses Neighbor Discovery for the same purpose [43]. Hosts usually obtain their address and other configuration, such as the default router and name server, automatically from the Dynamic Host Configuration Protocol (DHCP) [44]. These protocols are the seam between the link layer and the internet layer of Fig. 2.

D. Forwarding and Routing

Two jobs inside a router are worth separating. *Forwarding* is the per-packet decision: look up the destination in a table, using the longest matching prefix, and send the packet out of the chosen interface. *Routing* is the slower process that builds the table. The two are often called the *data plane* and the *control plane* [45].

Within a single organization's network, routers run an interior routing protocol. In a *link-state* protocol such as OSPF, every router in an area learns that area's full topology and computes shortest paths itself [46]. In a *distance-vector* protocol such as RIP, each router tells its neighbors only its distance to each destination, and routes emerge from repeated exchanges [47]. Between organizations, each of which forms an *autonomous system*, the Border Gateway Protocol (BGP) exchanges reachability together with the full list of autonomous systems along each path, which lets each operator apply its own policy about which routes to accept and announce [48]. BGP is less a shortest-path algorithm than a negotiation among independent networks, and its policies, not distances, largely determine the path a packet takes across the Internet.

E. Time to Live, ICMP, and `traceroute`

A routing mistake could send a packet around a loop forever, so every IPv4 datagram carries a *time to live* (TTL) field that each router decrements; "if this field contains the value zero, then the datagram must be destroyed" [34]. IPv6 calls the same field the hop limit [35]. The router that discards such a packet reports it with the Internet Control Message Protocol (ICMP), which also carries the echo request and reply used by `ping` [49]. The `traceroute` tool exploits both: it sends probes with TTL 1, 2, 3, and so on, and each router that discards one reveals itself in the ICMP report it returns [50], [51, Sec. 1]. A diagnostic tool is thus built entirely from a safety mechanism.

F. Packet Size Along the Path

Each link has a maximum transmission unit (MTU), and a path is limited by its smallest one. IPv4 routers may split a datagram that is too large, but fragmentation is costly and fragile, and IPv6 routers do not fragment at all; only the sending host may [35]. Hosts therefore discover the usable size with *path MTU discovery*: IPv4 hosts set the "Don't Fragment" bit, IPv6 packets are never fragmented by routers in any case, and in both the host

reduces its packet size when a router reports, by ICMP, that a packet was too big [52], [53]. A network that blocks those ICMP messages silently breaks large transfers, a classic and baffling failure [53].

Key point. Addresses come in prefixes, and routers forward by the longest match. Interior protocols compute shortest paths; BGP negotiates policy between independent networks. TTL, ICMP, and path MTU discovery keep the whole system safe and diagnosable.

VII. Names: The Domain Name System

People and programs use names such as `www.example.org`; IP needs addresses. The Domain Name System (DNS) maps one to the other [54], [55].

A. A Distributed, Cached Database

DNS is a hierarchical, distributed database. The name space is a tree, read from right to left: the root, then top-level domains such as `org`, then `example.org`, and so on. Each part of the tree, a *zone*, is served by its own authoritative name servers, so no single organization holds the whole database. A program's request goes to a *resolver*, which, if it does not already know the answer, asks a root server where to find `org`, asks an `org` server where to find `example.org`, and finally asks that zone's server for the address [54]. Answers are *resource records*, such as an A record for an IPv4 address [55] or an AAAA record for an IPv6 address [56], and each carries a time to live that tells resolvers how long they may cache it. Caching is what makes the system fast and scalable: many queries are answered from a nearby cache without reaching the authoritative servers at all. Mockapetris and Dunlap described the design choices, and the experience behind them, in 1988 [57].

B. What It Means for Programs

For a C program, DNS is hidden behind `getaddrinfo()`, but it remains a network operation: it takes time, it can fail, and its answer can change. On the sandboxed machine used to prepare this paper, which can reach the Internet only through an HTTPS proxy and cannot reach its configured name servers, a lookup of `example.com` failed with the message "Name or service not known", while lookups of local names succeeded. A portable program treats name resolution like any other network call: it checks the result, reports `gai_strerror()` on failure, does not assume that a name yields only one address, and resolves again rather than caching an address forever.

C. Securing Names

Classic DNS has no protection against forged answers or eavesdropping [58], [59]. DNSSEC adds digital signatures so that resolvers can verify that answers come from the zone's owner [58], and DNS over TLS [59] and DNS over HTTPS [60] encrypt queries between a program and its resolver.

Key point. DNS is a tree of independently served zones, made fast by caching with time limits. For programs, a name lookup is a network operation that can be slow, can fail, and can return several addresses.

VIII. End to End: TCP and UDP

If IP may lose, duplicate, and reorder packets, who makes communication reliable? The answer comes from one of the most influential arguments in system design.

A. The End-to-End Argument

Saltzer, Reed, and Clark observed that a function such as reliable delivery "can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system" [61]. Checking every link along the way does not remove the need for an end-to-end check, because errors can also occur inside the routers, in the receiving host, or in the application. Implementing reliability in the network is therefore at best a performance optimization. The Internet puts it in the hosts, in the transport layer, and keeps the network simple.

B. UDP: Almost Nothing Added

The User Datagram Protocol adds very little to IP: an 8-byte header with source and destination ports, a length, and a checksum [62]. Ports let several programs on one host share an address. UDP keeps IP's best-effort service and its message boundaries: each datagram sent is delivered whole or not at all. It suits applications that prefer timeliness to completeness, such as voice and video, or that build their own reliability on top.

C. TCP: A Reliable Byte Stream

The Transmission Control Protocol builds a reliable, ordered *byte stream* between two endpoints [63]. A connection begins with a three-way handshake that agrees on initial sequence numbers. Every byte is numbered, the receiver acknowledges what has arrived, and the sender retransmits what is not acknowledged in time. *Flow control* keeps the sender from overrunning the receiver's buffer. *Congestion control* keeps senders from overrunning the network. When it was missing, the early Internet suffered a series of "congestion collapses"; in the first, in October 1986, throughput between Lawrence Berkeley Laboratory and the University of California at Berkeley fell from 32 kbit/s to 40 bit/s. The collapses continued until Jacobson introduced the algorithms, based on the principle of "conservation of packets", that TCP still uses [64], [65].

Two quantities explain most of TCP's performance. The *round-trip time* (RTT) is the time for data to reach the receiver and an acknowledgment to return. The *bandwidth-delay product* is the amount of data that must be in flight to keep a link busy: on a 1 Gbit/s path with a 50 ms RTT it is 50 million bits, about 6.25 megabytes, which is why TCP must keep large amounts of unacknowledged data outstanding, and why a new connection, which starts

TABLE II
NUMBERS TO KEEP IN MIND: MINIMUM ROUND-TRIP TIMES

Path	Distance	Min. RTT
Same machine (loopback, measured)	—	0.06 ms
London–Frankfurt	638 km	6.4 ms
New York–Los Angeles	3,936 km	39.4 ms
New York–London	5,570 km	55.7 ms
London–Sydney	16,994 km	169.9 ms

slowly and probes for capacity, takes several round trips to reach full speed [65], [66].

Table II gives a sense of scale. The distances are great-circle distances between city centers, and the times are the minimum round trip at 5 μs per kilometre, the delay of light in fibre [17]. They are physical lower bounds: real cables follow longer routes, and every router and queue adds more. The loopback figure is the request-and-reply time measured in Section XIII. No amount of bandwidth reduces these numbers, which is why the number of round trips a protocol needs often matters more than the speed of the link.

D. Ports

A transport connection is identified not by addresses alone but by two addresses and two *ports*, 16-bit numbers that select a program on each host. Port numbers are divided into three ranges: system ports from 0 to 1023, user ports from 1024 to 49151, and dynamic or private ports from 49152 to 65535 [67]. A server listens on a known port, such as 443 for HTTPS; a client's operating system normally picks a temporary port for its side of each connection.

E. The Life of a Connection

A TCP connection ends as carefully as it begins. Each side closes its direction of the stream separately, and the side that closes first then waits in the *TIME-WAIT* state for twice the maximum segment lifetime, which the standard takes to be two minutes, before forgetting the connection [63]. The wait ensures that stray segments from the old connection cannot be mistaken for a new one using the same addresses and ports. It has a practical consequence for servers, discussed in Section XIII: a server that closes connections itself leaves its port briefly occupied after it exits.

F. Queues and Delay

Packet switching puts queues in every router, and queues add delay. Memory became cheap enough that many devices were built with very large buffers, which TCP's congestion control, designed to keep increasing its rate until packets are lost, then fills. Gettys and Nichols describe the result as *bufferbloat*, "the existence of excessively large and frequently full buffers inside the network" [68]: a link can show high throughput and, at the same time, hundreds of milliseconds or even seconds of needless latency. Throughput and latency are different properties, and a network can be good at one and bad at the other.

TABLE III
THE CENTRAL SOCKET CALLS

Call	Effect
`getaddrinfo`	Translate a host name and service into socket addresses
`socket`	Create an endpoint of a given family and type
`bind`	Attach a local address and port
`listen`	Mark a stream socket as accepting connections
`accept`	Take the next connection, as a new socket
`connect`	Open a connection to a remote address
`send`, `recv`	Transfer bytes or datagrams
`close`	Release the endpoint

G. Newer Transports

The transport layer continues to evolve above the narrow waist. QUIC, standardized in 2021, provides reliable streams over UDP with encryption built in, and avoids having a single lost packet delay every stream on the connection [69]. That it could be deployed at all, without changing a single router, is the hourglass doing its job.

Key point. Reliability belongs at the ends. UDP keeps IP's datagrams; TCP turns them into a reliable byte stream with flow and congestion control. Its speed is governed by the round-trip time and the bandwidth-delay product.

IX. SOCKETS: THE PORTABLE ABSTRACTION

C programs reach the network through the *sockets* interface, introduced in 4.2BSD Unix in 1983 [70] and now part of POSIX [71]. A socket is an endpoint of communication, represented, like an open file, by a small integer called a file descriptor. Its design mirrors the layers below: the program chooses an address family (IPv4 or IPv6), a socket type (a byte stream or datagrams), and a protocol (normally implied by the other two), and the operating system implements everything beneath.

Table III lists the central calls. A server creates a socket, binds it to a local address and port, marks it as listening, and accepts connections, each of which yields a new socket. A client creates a socket and connects it to a server. Both then transfer data with `send()` and `recv()` and finish with `close()`.

The interface is portable in the same sense as C itself. A program written against it runs over Ethernet or Wi-Fi, IPv4 or IPv6, without change, because the abstraction exposes only what the layers below guarantee. That is also its trap: it exposes *exactly* what they guarantee, which is less than many programs assume.

Key point. Sockets give C programs a file-like handle on the transport layer. They hide the layers below, but not their limits.

X. Serving Many Connections

By default, socket calls *block*: `accept()` waits for a connection and `recv()` waits for data. That is convenient for a client talking to one server, but a server must talk to many clients at once, and a call that waits for one client stops it from serving the others.

A. One Thread per Connection

The simplest answer is to give each connection its own process or thread, each blocking independently. This works well for modest numbers of clients, but every thread costs memory for its stack and time for the operating system to switch between them. Around 1999 Kegel posed the question of how to handle ten thousand clients at once on a single server, the "C10K problem", and surveyed the answers then emerging [72].

B. Readiness Notification

The alternative is for one thread to wait for *any* of many sockets to become ready, and then serve whichever is. POSIX provides `select()` and `poll()` for this [73], [74]: the program passes a list of descriptors and the events it cares about, and the call returns when at least one is ready. The following run shows a single thread, using one `poll()` loop, serving five clients that connected at once but sent their requests at different times:

```
accepted connection 1 (now watching 2 sockets)
...
accepted connection 5 (now watching 6 sockets)
socket 5 ready: got "c4", echoing
socket 4 ready: got "c3", echoing
socket 3 ready: got "c2", echoing
socket 2 ready: got "c1", echoing
socket 1 ready: got "c0", echoing
```

The last client to connect happened to send first, and was served first: the server follows readiness, not the order of arrival, and in this run it never waited on an idle client.

`select()` and `poll()` must examine the whole list on every call, which becomes expensive with many thousands of connections [75]. Operating systems therefore added interfaces that keep the list inside the kernel and report only the descriptors that are ready: `epoll` on Linux, which "scales well to large numbers of watched file descriptors" [76], and `kqueue` on the BSD systems [75]. They are not part of POSIX, so portable servers usually reach them through a small library layer, the same confinement of platform-specific code recommended in the C paper of this series.

C. Completion Notification

Readiness interfaces still require one system call to learn that a socket is ready and another to perform the transfer. Linux's `io_uring` takes a different approach: the program places requests in a queue shared with the kernel, and the kernel reports each operation when it has *completed* [77], [78]. The progression from blocking calls to readiness to completion mirrors the hardware story of the earlier papers: the abstraction remains the same, sending and receiving bytes, while the mechanism is adapted to make the expensive part, crossing between program and kernel, as rare as possible.

Key point. Blocking calls suit one conversation; servers wait on many sockets at once. `poll()` is portable; `epoll`, `kqueue`, and `io_uring` scale further but belong behind a portable interface.

XI. Securing the Connection: TLS

Everything described so far assumes that the parties on the path are honest. They need not be. A packet crosses links and routers owned by others, any of which could read it, change it, or answer in place of the intended server. Transport Layer Security (TLS) runs over TCP and restores what the path cannot promise.

A. What TLS Provides

TLS 1.3 states its three goals plainly [79]. *Authentication*: "the server side of the channel is always authenticated". *Confidentiality*: data sent over the channel is "only visible to the endpoints". *Integrity*: data "cannot be modified by attackers without detection". A handshake at the start of the connection agrees on fresh keys, and every record afterwards is encrypted and protected against modification with them.

B. How the Server Proves Who It Is

Encryption alone is not enough: an encrypted connection to an impostor is still a connection to an impostor. The server therefore presents a *certificate*, a signed statement binding a public key to a name, in the X.509 format [80]. The certificate is signed by a certificate authority, whose own certificate may be signed by another, in a chain that ends at a *trust anchor* the client already holds. The client must check two separate things: that the chain leads to a trusted anchor, and that the name in the certificate matches the name the client intended to reach. "The client checks each of its reference identifiers against the server's presented identifiers" [81]. The second check is the one most often forgotten.

C. The Most Common Mistake

In 2012 Georgiev and colleagues examined widely used software outside web browsers and concluded that "SSL certificate validation is completely broken in many security-critical applications and libraries" [82]. Much of the damage came from libraries whose defaults, or whose interfaces, made it easy to skip a check. The problem persists because the checks are separate calls: in OpenSSL, for example, the name is checked only if the program supplies one, and "if name is NULL or the empty string ... name checks are not performed" [83]. Section XIII shows the consequence in a real run.

Key point. TLS provides authentication, confidentiality, and integrity, but authentication works only if the client checks both that the certificate chain is trusted and that the certificate names the intended server.

XII. Terms, Derived

Networking has a large vocabulary, and much of it is taught as definitions to remember. Most terms are easier to keep straight once it is clear which part of the structure they describe. Table IV gathers the most commonly confused ones, and the paragraphs below explain the pairs that are most often mixed up.

A. Bandwidth, Baud, and Bit Rate

To an engineer, *bandwidth* is a width in hertz; to a programmer, it is a data rate in bits per second. The two are linked by Shannon's formula, which converts the first into the most the second can be. *Baud* counts signal changes per second; the *bit rate* is the baud multiplied by the number of bits each change carries, so a link signalling at 1000 baud with 16 distinguishable levels carries 4000 bit/s.

B. Throughput and Goodput

Throughput counts everything delivered; *goodput* excludes what was lost or retransmitted [85], and in the sense application programmers usually mean, it counts only what the application asked for. In that sense the difference also includes headers and acknowledgments. A full 1500-byte IPv4 packet carrying TCP holds at most 1460 bytes of application data when neither header has options, so even a perfect link delivers less goodput than throughput.

C. Frames, Packets, Datagrams, and Segments

These are not different kinds of data but the same data seen at different layers of Fig. 2. The Internet host requirements give each a precise meaning [32]: a TCP segment travels inside an IP datagram, which travels as one or more packets, each inside a link-layer frame. The names simply say which layer's header is being discussed.

D. MTU and MSS

The MTU belongs to the link: the largest packet it can carry. The MSS belongs to TCP: the largest payload one segment may carry. The value a host advertises to its peer "should be equal to the effective MTU minus the fixed IP and TCP headers" [63]. On Ethernet, with an MTU of 1500 bytes and 20-byte IPv4 and TCP headers, the MSS is 1460 bytes.

E. Hubs, Switches, Routers, and Load Balancers

A device's name says which header it reads. A *hub* reads none, repeating bits to every port. A *bridge* or *switch* reads the link-layer header [88]; a *router* reads the IP header. Load balancers are named the same way: a "layer-4" load balancer spreads connections using only addresses and ports [89], [90], while a "layer-7" load balancer looks inside the application data, for example at an HTTP request, before choosing a server. The early Internet papers used *gateway* for what is now called a router [33]; today the word usually means a device that translates between different protocols or networks.

Key point. Most networking terms name either a unit of data at some layer, a measure of some part of the path, or a device that reads some layer's header. Asking "which layer, and which part of the structure?" turns definitions into derivations.

XIII. Canonical Forms for Network Code

The C paper of this series defined a canonical form as a way of writing a construct that stays inside what is guaranteed, translates predictably, and makes mistakes detectable. Network code needs the same discipline, with one addition: the other end of a connection is a separate program, on a separate machine, possibly hostile, and it may fail at any moment. The forms below follow from one rule, *rely on nothing the layers below do not promise*. Each is illustrated by a small C99 program run on Linux over the loopback interface. Loopback never loses, reorders, or meaningfully delays a packet, so these runs show how the programming interface behaves, not how a real network misbehaves; where delay matters, it was injected in software and is labeled as such.

A. Byte Order

IP and the protocols above it transmit multi-byte integers with "the most significant octet ... transmitted first" [34], the order that the sockets interface calls *network byte order* [71]. Hosts may store integers either way. On the x86-64 machine used here, the address 192.168.0.1, held in a 32-bit integer, is stored in memory as `01 00 a8 c0`; after conversion with `htonl()` it becomes `c0 a8 00 01`, the order required on the wire. The canonical form converts every multi-byte field at the boundary: `htons()` and `htonl()` on the way out, `ntohs()` and `ntohl()` on the way in, or, for fields of any width, the shift-and-OR technique of the C paper, which is correct on every host by construction.

B. Wire Formats, Not Structures

It is tempting to send a C structure directly with `send(s, &msg, sizeof msg, 0)`. That transmits the compiler's layout, including padding between fields and the host's byte order, neither of which the other end is guaranteed to share. The canonical form defines the wire format byte by byte, writes each field into a byte buffer explicitly, and reads it back the same way. The structure is the program's private representation; the byte sequence is the protocol.

C. Message Boundaries

TCP delivers a stream of bytes, not a sequence of messages [63]. The following run shows the difference. A client called `send()` three times, with five bytes each time, over TCP and then over UDP:

TABLE IV
COMMONLY CONFUSED NETWORKING TERMS, BY WHAT THEY DESCRIBE

Term	What it names	Where it lives
Bandwidth (Hz)	Width of the frequency band a channel passes	The physical channel (Sec. II)
Bandwidth (bit/s)	Informal name for a link's data rate	The link; bounded by capacity
Baud	Symbols per second, the reciprocal of the shortest signal element's duration [84]	The physical signal
Bit rate	Bits per second: baud times bits per symbol	The physical signal
Throughput	All data delivered per second, headers and retransmissions included	Any layer
Goodput	Data delivered per second, minus any lost or retransmitted [85]; in application terms, useful payload only	The application
Latency	Time for data to travel from one end to the other	The path
Round-trip time	Time to the other end and back	The path, as seen by TCP
Jitter	Variation in delay from packet to packet [86]	The path, under load
Frame	"The unit of transmission in a link layer protocol" [32]	Link layer
Packet, datagram	The unit handed between internet and link layers; the unit of IP transmission [32]	Internet layer
Segment	"The unit of end-to-end transmission in the TCP protocol" [32]	Transport layer (TCP)
MTU	Largest packet a link can carry	Link layer
MSS	Largest TCP payload: the MTU minus IP and TCP headers [63], [87]	Transport layer
Hub	Repeats bits to every port	Physical layer
Bridge, switch	Forwards frames by link-layer address [88]	Link layer ("Layer 2")
Router	Forwards packets by IP address, longest prefix first	Internet layer ("Layer 3")
Port	16-bit number selecting a program on a host [67]	Transport layer
Socket	A program's handle on one transport endpoint	The operating system interface
Connection	Identified by protocol, two addresses, and two ports	Transport layer
L4 load balancer	Distributes connections by transport headers	Transport layer
L7 load balancer	Distributes requests by application content, such as a URL	Application layer

```
TCP: three send() calls of 5 bytes each
  recv() returned 15 bytes: HELLOWORLDAGAIN
UDP: three sendto() calls of 5 bytes each
  recv() returned 5 bytes: HELLO
  recv() returned 5 bytes: WORLD
  recv() returned 5 bytes: AGAIN
```

Over TCP, the three messages arrived merged into a single receive; on another occasion they might arrive split differently. Over UDP, each datagram arrived separately. A TCP program must therefore mark message boundaries itself. The canonical form is *framing*: precede each message with its length, in network byte order, or end it with a delimiter that cannot appear inside it, and have the receiver read exactly that many bytes.

D. Partial Transfers

Neither sending nor receiving is guaranteed to transfer as many bytes as requested. POSIX allows a non-blocking write to "write what it can and return the number of bytes written", and a receive returns whatever data is available [71]. On Linux, a non-blocking TCP socket asked to send 8 MiB at once accepted part of it and then, with its buffer full, refused more:

```
call 1: asked 8388608 bytes, send() accepted 3919467
call 2: send() = -1, errno = EAGAIN (buffer full)
```

Blocking sockets can also return early, for example when interrupted by a signal [71]. The canonical form wraps every transfer in a loop that continues until the whole buffer is done, retries on `EINTR`, and treats a zero return from `recv()` as the peer's orderly shutdown rather than as data [71]:

```
int send_all(int s, const void *buf, size_t n) {
  const char *p = buf;
  while (n > 0) {
    ssize_t k = send(s, p, n, 0);
    if (k < 0) { if (errno == EINTR) continue;
                 return -1; }
    p += k; n -= (size_t)k;
  }
  return 0;
}
```

E. Small Writes and Latency

TCP tries to avoid sending many tiny packets. Nagle's algorithm holds back a small segment while earlier data is still unacknowledged [91], and receivers commonly delay acknowledgments in the hope of combining them with a reply [32]. Together they can stall a common pattern: a client writes a request in two pieces and then waits for the answer. Fifty such exchanges over loopback gave:

```
two writes, Nagle on (default)    43.105 ms per request
two writes, TCP_NODELAY on         0.059 ms per request
one write (header+body together)   0.059 ms per request
```

The two-write pattern was about 700 times slower, because each second write waited for a delayed acknowledgment. Disabling Nagle's algorithm with the `TCP_NODELAY` option [92] removed the delay, but so did the simpler fix. The canonical form assembles each complete message in a buffer and writes it with one call, which is both faster and independent of socket options.

F. Round Trips

Because each round trip costs at least the delay of Table II, a protocol that waits for a reply after every small request is limited by latency, not bandwidth. To show the effect without a real long-distance path, a test server was made to wait 10 ms before each reply, a delay injected in software to stand in for the network. Fifty items were then requested in two ways:

```
50 requests, one round trip each:   525.0 ms
50 items in one batched request:     10.6 ms
```

The sequential version paid the delay fifty times; the batched version paid it once. The canonical form designs protocols to need few round trips: batch requests, send several before waiting for replies where the protocol allows it, and avoid exchanges whose only purpose is confirmation.

G. Names and Address Families

A host name may correspond to IPv4 addresses, IPv6 addresses, or both, and a portable program must not assume either. The canonical form resolves names with `getaddrinfo()` [93], requesting `AF_UNSPEC`, and tries each returned address in turn until one works. Asked for the addresses a server should listen on, the system used here returned both families:

```
IPv4  0.0.0.0
IPv6  ::
```

Clients that try addresses one after another can be slow when the first family is broken, so modern implementations start a connection on the preferred family and, if it has not succeeded after a short delay, race further attempts on the other family, an approach known as Happy Eyeballs [94]. Name resolution itself can fail, as Section VII showed, and the canonical form reports the error from `gai_strerror()` rather than proceeding with no address.

H. Errors, Timeouts, and Shutdown

Every socket call can fail, and network failures are normal events, not exceptional ones. The canonical form checks every return value. It sets a timeout on every operation that waits for the peer, with `poll()` or the `SO_RCVTIMEO` option [95], because a peer that vanishes without closing the connection otherwise leaves the program waiting indefinitely. And it prevents the signal `SIGPIPE`, which by default terminates a process that writes to a connection the peer has closed [71], by ignoring the signal or passing the `MSG_NOSIGNAL` flag, part of POSIX since 2008 [71], so that the error is reported as a return value instead.

I. Server Sockets

A server that closes connections first leaves those connections in TIME-WAIT, and while they remain, a restarted server cannot bind the same port. The remedy is the `SO_REUSEADDR` option [96], [95], but on Linux, as the following trials show, it helps only if the *original* listening socket set it too. Two trials over loopback, in which a server accepted a connection, closed it, exited, and was immediately restarted, show the effect:

```
first server did not set SO_REUSEADDR:
  restart without SO_REUSEADDR: Address already in use
  restart with SO_REUSEADDR:    Address already in use
first server set SO_REUSEADDR:
  restart without SO_REUSEADDR: Address already in use
  restart with SO_REUSEADDR:    bound
```

The canonical form sets `SO_REUSEADDR` on every listening socket before calling `bind()`, so that the server can always be restarted at once.

J. Datagrams

UDP leaves to the application everything that TCP would otherwise do. A datagram cannot exceed what the IP length field allows: over IPv4 the largest UDP payload is 65,535 bytes minus a 20-byte IP header and an 8-byte UDP header. The system used here accepted a 65,507-byte datagram and rejected a 65,508-byte one:

```
sendto(65507 bytes): accepted
sendto(65508 bytes): Message too long
```

Datagrams that large would in any case be fragmented on almost every path. The IETF's UDP usage guidelines [97] therefore give the canonical forms: keep each datagram small enough to avoid fragmentation, expect datagrams to be lost, duplicated, and reordered, and limit the sending rate so that the application does not congest the network, since UDP itself does nothing to prevent it.

K. Closing Cleanly

`close()` releases a socket in both directions at once. A program that has finished sending but still expects a reply should instead call `shutdown()` with `SHUT_WR` [98], which ends its outgoing direction, so that the peer reads an orderly end of stream, while the incoming direction remains open. Protocols that signal "end of request" by closing the sending direction depend on this half-close.

L. Untrusted Input

Everything received from the network is untrusted. A length field read from a message is only a claim, and copying that many bytes into a fixed buffer without checking is one of the oldest ways to compromise a program. The canonical form validates every field against the protocol's limits before using it, and rejects a message that fails. This also tempers an old maxim. Postel's *robustness principle*, "be conservative in what you do, be liberal in what you accept from others" [99], helped early implementations interoperate, but tolerating malformed input lets errors spread and invites attack, and a document published by the Internet Architecture Board now argues that tolerating unexpected input harms interoperability over time, and recommends actively maintaining protocols instead [100]. Where confidentiality or integrity matters,

the canonical form uses an established secure transport such as TLS 1.3 [79] rather than inventing its own, and verifies it as described next.

M. Verifying Identity

A TLS client is only as safe as its checks (Section XI). To show the difference, a test certificate authority was created, it issued a certificate for the name `server.example`, and a TLS server on loopback presented that certificate. A small C client built on OpenSSL then connected four times, using three levels of checking, each time believing it was reaching the name shown (output abridged):

```
full,      server.example: CONNECTED (TLSv1.3)
full,      bank.example:   REFUSED, hostname mismatch
chainonly, bank.example:   CONNECTED (TLSv1.3)
no-verify:                 CONNECTED (TLSv1.3)
```

With full verification, the client accepted the genuine server and refused the same server when it had intended to reach `bank.example`. With the chain checked but not the name, it connected happily to a server whose certificate was perfectly valid, but for a different name: exactly the failure Georgiev and colleagues found in deployed software [82]. With no verification it would talk to anyone. In all three successful cases the connection was encrypted with TLS 1.3; encryption was never the problem. The canonical form enables peer verification, loads the trust anchors, and always supplies the expected host name:

```
SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);
SSL_CTX_load_verify_locations(ctx, ca_file, NULL);
...
SSL_set_tlsext_host_name(ssl, host);  /* SNI       */
SSL_set1_host(ssl, host);   /* check the NAME too */
```

Key point. Rely on nothing the layers below do not promise: convert byte order, define wire formats explicitly, frame messages, loop over partial transfers, write whole messages at once, resolve names for both families, check every result, time out every wait, and validate every input, and verify the peer's identity, name included.

XIV. Putting It Together: A Small Protocol

The canonical forms of Section XIII are easier to judge together than one by one. This section presents a complete, if small, client and server, 134 lines of C99 in three files, that apply all of them. The protocol is deliberately simple: the client sends text, and the server replies with the same text in upper case.

A. The Wire Format

Every message, in either direction, is a 4-byte length N, most significant byte first, followed by N bytes of payload. N may not exceed 1024. Defining the format byte by byte, rather than as a C structure, fixes it independently of any compiler or host.

B. The Shared Helpers

A header file, `proto.h`, holds everything both programs need, and each function corresponds to one canonical form:

```
static inline void put_u32(uint8_t *b, uint32_t v) {
  b[0] = (uint8_t)(v >> 24); b[1] = (uint8_t)(v >> 16);
  b[2] = (uint8_t)(v >> 8);  b[3] = (uint8_t)v;
}
/* One message = one send: header and payload are
   assembled first, then sent with send_all(). */
static inline int send_msg(int s, const void *d,
                           uint32_t len) {
  uint8_t buf[4 + MAX_PAYLOAD];
  if (len > MAX_PAYLOAD) return -1;
  put_u32(buf, len); memcpy(buf + 4, d, len);
  return send_all(s, buf, 4u + len);
}
/* Payload length; 0 on orderly close; -1 on error
   or invalid input. */
static inline long recv_msg(int s, uint8_t *out) {
  uint8_t hdr[4]; int r = recv_all(s, hdr, 4);
  if (r <= 0) return r;
  uint32_t len = get_u32(hdr);
  if (len > MAX_PAYLOAD) return -1;  /* untrusted */
  if (len > 0 && recv_all(s, out, len) <= 0)
    return -1;
  return (long)len;
}
```

`put_u32` and its inverse `get_u32` fix the byte order by construction. `send_all` and `recv_all` are the loops of Section XIII, retrying on `EINTR` and distinguishing an orderly close from an error. `send_msg` assembles each complete message before sending it, avoiding the small-write stall. `recv_msg` implements the framing, and checks the length against the protocol's limit *before* reading the payload. A seventh helper, `set_timeout()`, bounds every send and receive; both programs set it to five seconds.

C. The Server and the Client

The server resolves its listening addresses with `getaddrinfo()` in passive mode and tries each one, sets `SO_REUSEADDR` before `bind()`, ignores `SIGPIPE` so that a vanished client produces an error rather than termination, applies the timeouts, and closes any connection that sends an invalid message. The client resolves the server's name, tries each address in turn, sends each word as one message, and ends with `shutdown(SHUT_WR)` before `close()`. Both compile without warnings under `-std=c99 -Wall -Wextra -Wconversion`.

D. A Run, Including an Attack

With the server running on port 5555, the client produced:

```
sent hello    got HELLO
sent canonical got CANONICAL
sent forms    got FORMS
```

A third program then played a hostile client. It connected and sent only a header claiming a payload of $2^{32}-1$ bytes, about 4 GB. The server read the 4-byte header, rejected the claimed length without reading or allocating any payload, closed the connection, and logged:

```
server: rejected a malformed or oversized message
```

TABLE V
SUMMARY OF CANONICAL FORMS FOR NETWORK CODE

Concern	What the network does not promise	Canonical form
Byte order	That the peer stores integers as this host does	Convert every multi-byte field at the boundary (`htonl`/`ntohl` or shifts and ORs)
Wire format	That the peer shares the compiler's layout	Define the format byte by byte; never send a structure directly
Boundaries	That TCP preserves message boundaries	Length-prefix or delimiter framing; read exactly the framed length
Partial transfer	That a call moves all requested bytes	Loop until complete; retry on `EINTR`; treat `recv() == 0` as shutdown
Small writes	That separate writes leave promptly	Assemble each message and send it with one call; use `TCP_NODELAY` only when needed
Addresses	That a host has one address family	`getaddrinfo` with `AF_UNSPEC`; try every result
Failures	That the peer stays reachable	Check every result; time out every wait; avoid `SIGPIPE`
Input	That the peer is honest or correct	Validate every field against protocol limits; reject malformed messages
Server sockets	That a port is free again as soon as a server exits	Set `SO_REUSEADDR` on every listening socket before `bind()`
Datagrams	That UDP delivers, orders, or paces anything	Small datagrams that avoid fragmentation; tolerate loss and duplication; limit the rate
Closing	That `close()` leaves a reply possible	`shutdown(SHUT_WR)` to end sending while still receiving
Concurrency	That one client at a time is enough	`poll()` portably; `epoll`/`kqueue`/`io_uring` behind a portable interface
Round trips	That latency is small	Batch requests; send several before waiting; avoid confirmation-only exchanges
Security	That anyone on the path is trustworthy	Use an established secure transport such as TLS
Identity	That an encrypted peer is the intended one	Verify the certificate chain *and* the host name

Nothing in the example is clever. Its robustness comes entirely from refusing to rely on anything the network does not promise: not the peer's byte order, not message boundaries, not complete transfers, not prompt replies, and not honest input.

Key point. A small set of helpers, one per canonical form, makes a protocol implementation correct by construction. The server survives a hostile client because it validates before it trusts.

XV. Discussion

The paper still leaves out a good deal. It treats routing protocols, the name system, and wireless links only in outline; each has a literature of its own. It does not cover multicast, software-defined networking, cellular networks, or the mechanisms for moving between IPv4 and IPv6, and it explains TLS only as far as a program must understand it in order to use it safely. The intent has been to show the structure that makes those topics fit together, not to survey them. The demonstrations, too, have a stated limit. All ran over the loopback interface of one Linux machine, which never loses, reorders, or meaningfully delays packets, and whose kernel did not provide the facilities for emulating impairments at the packet level. They therefore show how the programming interface behaves, not how TCP responds to loss or long delay; where delay mattered, it was injected in software at the application level and is labeled as such.

The structure is the same one found beneath C. Physical media diverge as processors do, each with its own capacity, delay, and error behavior. Engineers named the differences with layer numbers and device categories, much as they named processor differences with CISC, RISC, and cache levels. A deliberately minimal common core, IP, makes interconnection possible, as the minimal machine makes portable C possible. A portable interface, sockets, sits on that core, and correct programs are written in canonical forms that rely only on what the core promises. In both cases the categories need not be memorized; they follow from the structure.

XVI. Conclusion

A network is a physical channel with finite rate, unavoidable delay, and occasional errors, shared packet by packet, organized in layers, and joined across all its varieties by one deliberately minimal protocol, with prefixes and routing protocols finding paths and a cached, distributed database turning names into addresses. Reliability is added at the ends, where the end-to-end argument says it must be, and the sockets interface gives C programs a portable handle on the result. That handle exposes exactly what the network promises, and no more. Programs that respect this, by converting byte order, framing messages, looping over partial transfers, writing whole messages, handling every address family and every failure, and trusting no input, are correct on every network. Those that do not may work in testing and fail in the field.

Appendix A
Seeing the Network

Much of what this paper describes can be observed directly with standard tools. Table VI lists the most useful; each is documented in its own manual page.

References

[1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 9th ed. Pearson, 2025.
[2] C. E. Shannon, "Communication in the presence of noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, Jan. 1949.
[3] M. Unser, "Sampling—50 years after Shannon," *Proceedings of the IEEE*, vol. 88, no. 4, pp. 569–587, Apr. 2000.
[4] H. Nyquist, "Certain topics in telegraph transmission theory," *Transactions of the American Institute of Electrical Engineers*, vol. 47, no. 2, pp. 617–644, Apr. 1928.

TABLE VI
TOOLS FOR OBSERVING THE NETWORK

Tool	What it shows
`ping` [101]	Whether a host answers ICMP echo, and the round-trip time
`traceroute` [50]	The routers along a path, revealed by probes of increasing TTL
`dig` [102]	DNS queries and answers, including records and their TTLs
`ss` [103]	A host's sockets and connections, including their TCP states
`tcpdump` [104]	The packets crossing an interface, header by header
Wireshark [105]	The same, decoded layer by layer in a graphical interface

[5] J. B. Johnson, "Thermal agitation of electricity in conductors," *Physical Review*, vol. 32, no. 1, pp. 97–109, Jul. 1928.

[6] H. Nyquist, "Thermal agitation of electric charge in conductors," *Physical Review*, vol. 32, no. 1, pp. 110–113, Jul. 1928.

[7] ITU-R, "Radio noise," Recommendation ITU-R P.372-18, Sep. 2026. [Online]. Available: https://www.itu.int/rec/R-REC-P.372/en

[8] R. V. L. Hartley, "Transmission of information," *Bell System Technical Journal*, vol. 7, no. 3, pp. 535–563, Jul. 1928.

[9] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, Jul. 1948.

[10] ——, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 4, pp. 623–656, Oct. 1948.

[11] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ, USA: Wiley-Interscience, 2006.

[12] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge, UK: Cambridge University Press, 2003. [Online]. Available: https://www.inference.org.uk/mackay/itila/

[13] T. Miya, Y. Terunuma, T. Hosaka, and T. Miyashita, "Ultimate low-loss single-mode fibre at 1.55 μm," *Electronics Letters*, vol. 15, no. 4, pp. 106–108, Feb. 1979.

[14] ITU-T, "Characteristics of a single-mode optical fibre and cable," Recommendation ITU-T G.652, Aug. 2024. [Online]. Available: https://www.itu.int/rec/T-REC-G.652-202408-I/en

[15] H. T. Friis, "A note on a simple transmission formula," *Proceedings of the IRE*, vol. 34, no. 5, pp. 254–256, May 1946.

[16] IEEE, "IEEE standard for Ethernet," IEEE Std 802.3-2022, Jul. 2022, clause 40.7.2 (1000BASE-T link segment).

[17] ITU-T, "One-way transmission time," ITU-T Recommendation G.114, May 2003. [Online]. Available: https://www.itu.int/rec/T-REC-G.114-200305-I/en

[18] IEEE, "IEEE standard for Ethernet," IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018), Jul. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9844436

[19] ——, "IEEE standard for information technology–telecommunications and information exchange between systems–local and metropolitan area networks–part 3: CSMA/CD access method and physical layer specifications–physical layer parameters and specifications for 1000 Mb/s operation over 4 pair of category 5 balanced copper cabling, type 1000BASE-T," IEEE Std 802.3ab-1999, 1999. [Online]. Available: https://www.ieee802.org/3/ab/index.html

[20] ——, "IEEE standard for information technology–telecommunications and information exchange between systems–LAN/MAN–specific requirements part 3: CSMA/CD access method and physical layer specifications–amendment: Physical layer and management parameters for 10 Gb/s operation, type 10GBASE-T," IEEE Std 802.3an-2006, Sep. 2006. [Online]. Available: https://www.ieee802.org/3/an/index.html

[21] ——, "IEEE standard for information technology–local and metropolitan area networks–part 3: CSMA/CD access method and physical layer specifications–media access control (MAC) parameters, physical layer, and management parameters for 10 Gb/s operation," IEEE Std 802.3ae-2002, 2002. [Online]. Available: https://www.ieee802.org/3/ae/index.html

[22] ——, "IEEE standard for information technology–local and metropolitan area networks–specific requirements–part 3: CSMA/CD access method and physical layer specifications amendment 4: Media access control parameters, physical layers, and management parameters for 40 Gb/s and 100 Gb/s operation," IEEE Std 802.3ba-2010, Jun. 2010. [Online]. Available: https://www.ieee802.org/3/ba/index.html

[23] ——, "IEEE standard for information technology–telecommunications and information exchange between systems local and metropolitan area networks–specific requirements part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specifications amendment 1: Enhancements for high-efficiency WLAN," IEEE Std 802.11ax-2021 (Amendment to IEEE Std 802.11-2020), May 2021. [Online]. Available: https://standards.ieee.org/ieee/802.11ax/7180/

[24] P. Baran, "On distributed communications: I. introduction to distributed communications networks," RAND Corporation, Santa Monica, CA, USA, Research Memorandum RM-3420-PR, Aug. 1964. [Online]. Available: https://www.rand.org/pubs/research_memoranda/RM3420.html

[25] D. W. Davies, "Proposal for a digital communication network," National Physical Laboratory, Tech. Rep., Jun. 1966. [Online]. Available: https://www.computerhistory.org/collections/catalog/102660191

[26] ——, "An historical study of the beginnings of packet switching," *The Computer Journal*, vol. 44, no. 3, pp. 152–162, 2001.

[27] R. M. Metcalfe and D. R. Boggs, "Ethernet: Distributed packet switching for local computer networks," *Communications of the ACM*, vol. 19, no. 7, pp. 395–404, Jul. 1976.

[28] V. Bharghavan, A. Demers, S. Shenker, and L. Zhang, "MACAW: A media access protocol for wireless LAN's," in *Proc. ACM SIGCOMM Conf. Communications Architectures, Protocols and Applications*, London, U.K., Aug. 1994, pp. 212–225.

[29] IEEE, "IEEE standard for information technology–telecommunications and information exchange between systems - local and metropolitan area networks–specific requirements - part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specifications," IEEE Std 802.11-2020 (Revision of IEEE Std 802.11-2016), Feb. 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9363693

[30] H. Zimmermann, "OSI reference model—The ISO model of architecture for open systems interconnection," *IEEE Transactions on Communications*, vol. 28, no. 4, pp. 425–432, Apr. 1980.

[31] ISO/IEC, "Information technology—Open systems interconnection—Basic reference model: The basic model," International Standard ISO/IEC 7498-1:1994, Nov. 1994. [Online]. Available: https://www.iso.org/standard/20269.html

[32] R. Braden, "Requirements for Internet hosts – communication layers," RFC 1122, IETF, RFC 1122, Oct. 1989. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1122

[33] V. G. Cerf and R. E. Kahn, "A protocol for packet network intercommunication," *IEEE Transactions on Communications*, vol. 22, no. 5, pp. 637–648, May 1974.

[34] J. Postel, "Internet protocol," RFC 791, IETF, RFC 791, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc791

[35] S. Deering and R. Hinden, "Internet protocol, version 6 (IPv6) specification," RFC 8200 (STD 86), IETF, RFC 8200, Jul. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8200

[36] D. D. Clark, "The design philosophy of the DARPA internet protocols," in *Proceedings of the ACM SIGCOMM '88 Symposium on Communications Architectures and Protocols*, Stanford, CA, USA, Aug. 1988, pp. 106–114.

[37] S. Deering, "Watching the waist of the protocol hourglass," Plenary talk, 51st IETF Meeting, London, U.K., Aug. 2001. [Online]. Available: https://www.ietf.org/proceedings/51/slides/plenary-1/index.html

[38] V. Fuller and T. Li, "Classless inter-domain routing (CIDR): The internet address assignment and aggregation plan,"

IETF, RFC 4632, Aug. 2006, bCP 122. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4632
[39] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, "Address allocation for private internets," IETF, RFC 1918, Feb. 1996, bCP 5. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1918
[40] P. Srisuresh and K. Egevang, "Traditional IP network address translator (traditional NAT)," IETF, RFC 3022, Jan. 2001. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3022
[41] T. Hain, "Architectural implications of NAT," IETF, RFC 2993, Nov. 2000. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2993
[42] D. C. Plummer, "An Ethernet address resolution protocol: Or converting network protocol addresses to 48.bit Ethernet address for transmission on Ethernet hardware," IETF, RFC 826, Nov. 1982, sTD 37. [Online]. Available: https://www.rfc-editor.org/rfc/rfc826
[43] T. Narten, E. Nordmark, W. Simpson, and H. Soliman, "Neighbor discovery for IP version 6 (IPv6)," IETF, RFC 4861, Sep. 2007. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4861
[44] R. Droms, "Dynamic host configuration protocol," IETF, RFC 2131, Mar. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2131
[45] E. Haleplidis, K. Pentikousis, S. Denazis, J. Hadi Salim, D. Meyer, and O. Koufopavlou, "Software-defined networking (SDN): Layers and architecture terminology," IRTF, RFC 7426, Jan. 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7426
[46] J. Moy, "OSPF version 2," IETF, RFC 2328, Apr. 1998, sTD 54. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2328
[47] G. Malkin, "RIP version 2," IETF, RFC 2453, Nov. 1998, sTD 56. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2453
[48] Y. Rekhter, T. Li, and S. Hares, "A border gateway protocol 4 (BGP-4)," IETF, RFC 4271, Jan. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4271
[49] J. Postel, "Internet control message protocol," IETF, RFC 792, Sep. 1981, sTD 5. [Online]. Available: https://www.rfc-editor.org/rfc/rfc792
[50] traceroute project, "traceroute(8) – print the route packets trace to network host," Linux manual page, 2026. [Online]. Available: https://man7.org/linux/man-pages/man8/traceroute.8.html
[51] G. Malkin, "Traceroute using an IP option," IETF, RFC 1393, Jan. 1993, historic; obsoleted by RFC 6814. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1393
[52] J. Mogul and S. Deering, "Path MTU discovery," IETF, RFC 1191, Nov. 1990. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1191
[53] J. McCann, S. Deering, J. Mogul, and R. Hinden, "Path MTU discovery for IP version 6," IETF, RFC 8201, Jul. 2017, sTD 87. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8201
[54] P. Mockapetris, "Domain names – concepts and facilities," IETF, RFC 1034, Nov. 1987, sTD 13. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1034
[55] ——, "Domain names – implementation and specification," IETF, RFC 1035, Nov. 1987, sTD 13. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1035
[56] S. Thomson, C. Huitema, V. Ksinant, and M. Souissi, "DNS extensions to support IP version 6," IETF, RFC 3596, Oct. 2003. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3596
[57] P. V. Mockapetris and K. J. Dunlap, "Development of the domain name system," in *Proc. ACM SIGCOMM Symp. Communications Architectures and Protocols*, Stanford, CA, USA, Aug. 1988, pp. 123–133.
[58] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS security introduction and requirements," IETF, RFC 4033, Mar. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4033
[59] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, and P. Hoffman, "Specification for DNS over transport layer security (TLS)," IETF, RFC 7858, May 2016. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7858
[60] P. Hoffman and P. McManus, "DNS queries over HTTPS (DoH)," IETF, RFC 8484, Oct. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8484
[61] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, Nov. 1984.
[62] J. Postel, "User datagram protocol," RFC 768, IETF, RFC 768, Aug. 1980. [Online]. Available: https://www.rfc-editor.org/rfc/rfc768
[63] W. Eddy, "Transmission control protocol (TCP)," RFC 9293 (STD 7), IETF, RFC 9293, Aug. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293
[64] V. Jacobson, "Congestion avoidance and control," in *Proceedings of the ACM SIGCOMM '88 Symposium on Communications Architectures and Protocols*, Stanford, CA, USA, Aug. 1988, pp. 314–329.
[65] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," RFC 5681, IETF, RFC 5681, Sep. 2009. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5681
[66] I. Grigorik, *High Performance Browser Networking*. O'Reilly Media, 2013. [Online]. Available: https://hpbn.co/
[67] M. Cotton, L. Eggert, J. Touch, M. Westerlund, and S. Cheshire, "Internet assigned numbers authority (IANA) procedures for the management of the service name and transport protocol port number registry," IETF, RFC 6335, Aug. 2011, bCP 165. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6335
[68] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, Jan. 2012.
[69] J. Iyengar and E. M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, RFC 9000, May 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9000
[70] S. J. Leffler, R. S. Fabry, and W. N. Joy, "A 4.2BSD interprocess communication primer," Computer Science Division, University of California, Berkeley, Berkeley, CA, USA, Tech. Rep. UCB/CSD-83-145, Jul. 1983, draft of July 27, 1983. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/1983/5451.html
[71] IEEE and The Open Group, "IEEE/Open Group standard for information technology–portable operating system interface (POSIX) base specifications, issue 8," IEEE Std 1003.1-2024 / The Open Group Base Specifications Issue 8, 2024. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/
[72] D. Kegel, "The C10K problem," Web page, last updated 5 Feb. 2014, 1999. [Online]. Available: https://www.kegel.com/c10k.html
[73] IEEE and The Open Group, "pselect, select – synchronous I/O multiplexing," The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024, System Interfaces, 2024. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/functions/select.html
[74] ——, "poll – input/output multiplexing," The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024, System Interfaces, 2024. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/functions/poll.html
[75] J. Lemon, "Kqueue: A generic and scalable event notification facility," in *Proc. FREENIX Track: 2001 USENIX Annual Technical Conference*. Boston, MA, USA: USENIX Association, Jun. 2001. [Online]. Available: https://www.usenix.org/legacy/event/usenix01/freenix01/lemon.html
[76] Linux man-pages project, "epoll(7) – I/O event notification facility," Linux Programmer's Manual, 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/epoll.7.html
[77] J. Axboe, "Efficient IO with io_uring," Technical note, version 0.4, Oct. 2019. [Online]. Available: https://kernel.dk/io_uring.pdf
[78] liburing project, "io_uring(7) – asynchronous I/O facility," Linux manual page (liburing), 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/io_uring.7.html
[79] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, IETF, RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446
[80] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 public key infrastructure

certificate and certificate revocation list (CRL) profile," RFC 5280, IETF, RFC 5280, May 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5280

[81] P. Saint-Andre and R. Salz, "Service identity in TLS," RFC 9525, IETF, RFC 9525, Nov. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9525

[82] M. Georgiev, S. Iyengar, S. Jana, R. Anubhai, D. Boneh, and V. Shmatikov, "The most dangerous code in the world: Validating SSL certificates in non-browser software," in *Proc. ACM Conf. Computer and Communications Security (CCS)*, Raleigh, NC, USA, Oct. 2012, pp. 38–49.

[83] OpenSSL Project, "SSL_set1_host(3) – SSL server verification parameters," OpenSSL 3.0 manual pages, 2026. [Online]. Available: https://docs.openssl.org/3.0/man3/SSL_set1_host/

[84] ITU-R, "Terms and symbols for information quantities in telecommunications," International Telecommunication Union, Recommendation ITU-R V.607-2, Jun. 1990, superseded by V.607-3 (2000), which was withdrawn in 2007. [Online]. Available: https://www.itu.int/rec/R-REC-V.607/en

[85] D. Newman, "Benchmarking terminology for firewall performance," RFC 2647, IETF, RFC 2647, Aug. 1999. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2647

[86] C. Demichelis and P. Chimento, "IP packet delay variation metric for IP performance metrics (IPPM)," RFC 3393, IETF, RFC 3393, Nov. 2002. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3393

[87] J. Postel, "The TCP maximum segment size and related topics," RFC 879, RFC Editor, RFC 879, Nov. 1983. [Online]. Available: https://www.rfc-editor.org/rfc/rfc879

[88] IEEE, "IEEE standard for local and metropolitan area networks–bridges and bridged networks," IEEE Std 802.1Q-2022 (Revision of IEEE Std 802.1Q-2018), Dec. 2022. [Online]. Available: https://ieeexplore.ieee.org/document/10004498

[89] P. Patel, D. Bansal, L. Yuan, A. Murthy, A. Greenberg, D. A. Maltz, R. Kern, H. Kumar, M. Zikos, H. Wu, C. Kim, and N. Karri, "Ananta: Cloud scale load balancing," in *Proc. ACM SIGCOMM*, Hong Kong, China, Aug. 2013, pp. 207–218.

[90] D. E. Eisenbud, C. Yi, C. Contavalli, C. Smith, R. Kononov, E. Mann-Hielscher, A. Cilingiroglu, B. Cheyney, W. Shang, and J. D. Hosein, "Maglev: A fast and reliable software network load balancer," in *Proc. 13th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, Santa Clara, CA, USA, Mar. 2016, pp. 523–535. [Online]. Available: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud

[91] J. Nagle, "Congestion control in IP/TCP internetworks," RFC 896, IETF, RFC 896, Jan. 1984. [Online]. Available: https://www.rfc-editor.org/rfc/rfc896

[92] Linux man-pages project, "tcp(7) – TCP protocol," Linux Programmer's Manual, man-pages 6.19, 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/tcp.7.html

[93] R. Gilligan, S. Thomson, J. Bound, J. McCann, and W. Stevens, "Basic socket interface extensions for IPv6," RFC 3493, IETF, RFC 3493, Feb. 2003. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3493

[94] D. Schinazi and T. Pauly, "Happy eyeballs version 2: Better connectivity using concurrency," RFC 8305, IETF, RFC 8305, Dec. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8305

[95] Linux man-pages project, "socket(7) – Linux socket interface," Linux Programmer's Manual, man-pages 6.19, 2026. [Online]. Available: https://man7.org/linux/man-pages/man7/socket.7.html

[96] IEEE and The Open Group, "setsockopt – set the socket options," The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024, System Interfaces (options defined in Sec. 2.10.16, Use of Options), 2024. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/functions/setsockopt.html

[97] L. Eggert, G. Fairhurst, and G. Shepherd, "UDP usage guidelines," RFC 8085 (BCP 145), IETF, RFC 8085, Mar. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8085

[98] IEEE and The Open Group, "shutdown – shut down socket send and receive operations," The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024, System Interfaces, 2024. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9799919799/functions/shutdown.html

[99] J. Postel, "DoD standard transmission control protocol," RFC 761, IETF, RFC 761, Jan. 1980. [Online]. Available: https://www.rfc-editor.org/rfc/rfc761

[100] M. Thomson and D. Schinazi, "Maintaining robust protocols," RFC 9413, Internet Architecture Board (IAB), RFC 9413, Jun. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9413

[101] iputils project, "ping(8) – send ICMP ECHO_REQUEST to network hosts," Linux manual page, 2026. [Online]. Available: https://man7.org/linux/man-pages/man8/ping.8.html

[102] Internet Systems Consortium, "dig – DNS lookup utility," BIND 9 Administrator Reference Manual, Manual Pages, 2026. [Online]. Available: https://bind9.readthedocs.io/en/latest/manpages.html

[103] iproute2 project, "ss(8) – another utility to investigate sockets," Linux manual page, 2026. [Online]. Available: https://man7.org/linux/man-pages/man8/ss.8.html

[104] The Tcpdump Group, "tcpdump(1) – dump traffic on a network," Manual page, 2026. [Online]. Available: https://www.tcpdump.org/manpages/tcpdump.1.html

[105] Wireshark Foundation, "Wireshark," Software, 2026. [Online]. Available: https://www.wireshark.org/